

12

MÁQUINAS QUE ENXERGAM: FILTRAGEM INTELIGENTE DE IMAGENS COM IA

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Kernel ou filtro sendo aplicado a uma imagem 7
Figura 2 - Comparação entre uma imagem original com ruído e sua versão suavizada com desfoque Gaussiano.....8
Figura 3 - Detalhes de bordas realçados por um filtro passa-alta como Sobel 9
Figura 4 - Matriz de imagem em tons de cinza 5×5 e matriz de kernel 3×3 11
Figura 5 - Ilustração passo a passo do kernel 3×3 deslizando sobre uma imagem 5×5, mostrando multiplicação e soma.....12
Figura 6 - Comparação visual entre convolução ‘válida’ (sem padding) e ‘mesma’ (com padding de zeros).....13
Figura 7 - O filtro Gaussiano como duas convoluções 1D consecutivas.....15
Figura 8 - Foto original *vs* resultado da aplicação do filtro de média 3×3.....18
Figura 9 - Kernel Gaussiano e efeito visual em uma imagem real 19
Figura 10 - Exemplo de imagem antes e depois da filtragem Laplaciana 20
Figura 11 - Filtro Sobel.....22
Figura 12 - Vários tipos de filtros podem ser desenvolvidos e aplicados a imagem..23
Figura 13 - Filtro da mediana 25
Figura 14 - Filtro da bilateral.....26
Figura 15 - Filtro guiado *vs* filtro bilateral 28
Figura 16 - Imagem no domínio espacial e no domínio da frequência 31
Figura 17 - Artefatos de bloco JPEG. As linhas pontilhadas vermelhas destacam os limites dos blocos 8 × 8 usados na compressão 34
Figura 18 - Filtro da média 36
Figura 19 - Filtro Gaussiano e Laplaciano.....38
Figura 20 - Filtro de Sobel.....40
Figura 21 - Filtro de nitidez.....41
Figura 22 - Removendo ruídos de imagens 43
Figura 23 - Melhorando a nitidez de imagens 45
Figura 24 - Melhorando imagens por meio de processamento no domínio da frequência.....46

LISTA DE QUADROS

Quadro 1 - Resumo dos principais parâmetros de convolução — tamanho do kernel, stride, padding e separabilidade.....16

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Filtro da média com NumPy.....35
Código-fonte 2 - Filtro Gaussiano e filtro Laplaciano.....37
Código-fonte 3 - Filtro de Sobel.....39
Código-fonte 4 - Filtro de nitidez 40
Código-fonte 5 - Remoção de ruídos 43
Código-fonte 6 - Melhoria de nitidez.....44
Código-fonte 7 - Remoção de padrões indesejáveis por meio de processamento no domínio da frequência.....46

FIAP

SUMÁRIO

1 MÁQUINAS QUE ENXERGAM: FILTRAGEM INTELIGENTE DE IMAGENS COM IA 6
1.1 Introdução 6

2 FUNDAMENTOS MATEMÁTICOS 11
2.1 Operação de convolução 11
2.1.1 Padding e efeitos de borda 12
2.1.2 Stride e redução de resolução 13
2.2 Linearidade e invariância a translação 14
2.2.1 Filtros separáveis 14
2.3 Correlação *versus* convolução 15

3 FILTROS LINEARES 17
3.1 Filtros de suavização (passa-baixa) 17
3.2 Filtros de nitidez (passa-alta) 19
3.3 Filtros de detecção de bordas 21
3.4 Comparação e casos de uso 22

4 FILTROS NÃO LINEARES 24
4.1 Filtro de mediana 24
4.2 Filtro bilateral 25
4.3 Filtro guiado 27
4.4 Filtragem não linear: pontos fortes e limitações 28
4.5 Considerações finais 29

5 FILTROS NO DOMÍNIO DA FREQUÊNCIA 30
5.1 O que é o domínio da frequência? 30
5.2 Transformada de Fourier 2D 31
5.3 Filtros passa-baixa e passa-alta 32
5.4 Filtragem: Domínio da frequência *versus* domínio espacial 33
5.5 Considerações finais 33

6 CASOS DE USO EM VISÃO COMPUTACIONAL E IMAGEM 34

7 LABORATÓRIO 35
7.1 Implementando filtros no NumPy 35
7.2 Convolução em Deep Learning (com PyTorch) 38
7.3 Exemplo de filtro personalizado: nitidez 40

8 EXEMPLOS PRÁTICOS E APLICAÇÕES 42
8.1 Redução de ruído em fotografias ruidosas 42
8.2 Aprimorando documentos para OCR com o filtro Sharpening 44
8.3 Filtragem no domínio da frequência para remover padrões 45

CONCLUSÃO 47

REFERÊNCIAS 48

FINAL

1 MÁQUINAS QUE ENXERGAM: FILTRAGEM INTELIGENTE DE IMAGENS COM IA

1.1 Introdução

Quando olhamos para uma imagem, nosso cérebro percebe com facilidade bordas, cores, texturas e padrões. No entanto, para um computador, uma imagem nada mais é do que uma matriz de números, intensidades de pixels que precisam ser analisadas matematicamente para extrair algum significado. É aqui que entra a filtragem de imagens.

A filtragem é uma das operações mais fundamentais no processamento de imagens e na Visão Computacional. Ela permite manipular, realçar e extrair características de imagens por meio da aplicação de operações matemáticas aos valores dos pixels.

Essencialmente, a filtragem de imagens refere-se ao processo de modificar ou aprimorar uma imagem, enfatizando certas estruturas ou suprimindo outras. Essas operações geralmente são realizadas com uma pequena matriz conhecida como kernel ou filtro, que percorre a imagem em uma janela deslizante. Em cada posição, o kernel é combinado com os valores de pixel subjacentes, na maioria das vezes por meio de convolução ou correlação, para produzir uma nova imagem onde certos padrões ou características são amplificados ou atenuados.

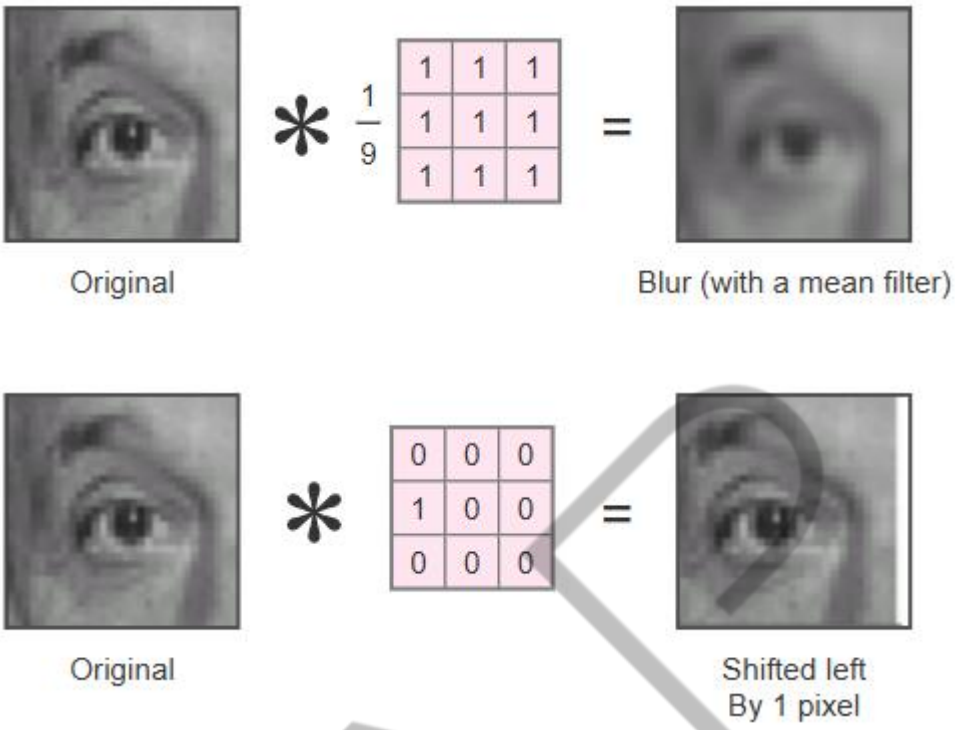


Figura 1 - Kernel ou filtro sendo aplicado a uma imagem
Fonte: Medium (2021), adaptado por FIAP (2025)

Esse conceito é fundamental para realizar uma ampla gama de tarefas de Visão Computacional. Por exemplo, antes de realizar a detecção de objetos em uma transmissão de câmera de segurança, podemos aplicar um filtro de remoção de ruído para eliminar interferências visuais indesejadas. Em imagens médicas, como ressonâncias magnéticas (MRI) ou tomografias (CT), filtros são frequentemente usados para realçar estruturas anatômicas ou remover detalhes irrelevantes. Em imagens de satélite, os filtros auxiliam na supressão de distorções atmosféricas e no destaque de características da superfície. Até mesmo os aplicativos do dia a dia — como os filtros do Instagram ou do TikTok — possuem princípios semelhantes, aplicados para suavizar tons de pele, aumentar o contraste ou realçar bordas com efeito artístico.

A filtragem pode ser amplamente categorizada em dois tipos principais: filtros passa-baixa e filtros passa-alta, com base no efeito que causam sobre as frequências da imagem.

Os filtros **passa-baixa** são projetados para suavizar imagens, reduzindo mudanças rápidas de intensidade — são frequentemente chamados de filtros de suavização ou desfoque. Eles suprimem componentes de alta frequência, como ruídos ou detalhes finos, sendo ideais para tarefas em que a redução de ruído é

crucial. Um exemplo comum é o filtro de suavização Gaussiano (Gaussian blur), que aplica uma média ponderada aos valores dos pixels, suavizando as transições entre regiões. Essa técnica é amplamente empregada antes da aplicação de algoritmos mais avançados — como detectores de bordas ou rastreadores de objetos — para garantir maior robustez contra detalhes espúrios.

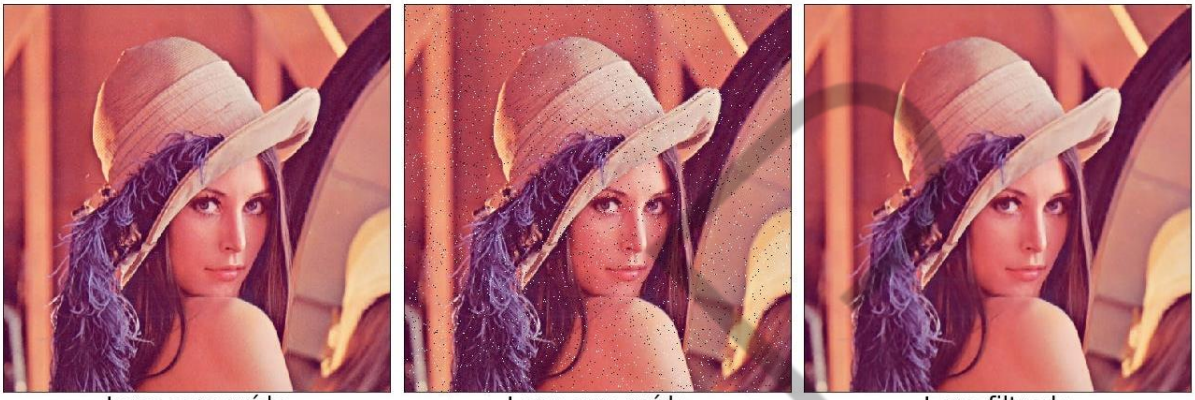


Figura 2 - Comparação entre uma imagem original com ruído e sua versão suavizada com desfoque Gaussiano
Fonte: Medium (2019)

Por outro lado, os filtros **passa-alta** fazem o oposto: eles enfatizam áreas onde os valores dos pixels mudam rapidamente, o que geralmente corresponde a bordas e detalhes finos da imagem. Esses filtros suprimem componentes de baixa frequência e são essenciais para tarefas que envolvem a detecção de bordas, cujo objetivo é identificar os contornos dos objetos. Filtros como os operadores de Sobel, Prewitt e Laplaciano são filtros passa-alta clássicos que ajudam a extrair bordas e linhas, tornando-se indispensáveis nas etapas iniciais dos pipelines de Visão Computacional.



Figura 3 - Detalhes de bordas realçados por um filtro passa-alta como Sobel
Fonte: LinkedIn (2024)

Embora pareça uma tarefa simples, a importância da filtragem não pode ser subestimada. Em praticamente todo fluxo de trabalho de Visão Computacional, seja em reconhecimento facial, monitoramento de tráfego, diagnóstico médico ou realidade aumentada, a filtragem de imagens atua como etapa de pré-processamento ou extração de características. Ela ajuda a preparar a imagem bruta para análises posteriores ou destaca estruturas de interesse.

Por exemplo, no reconhecimento óptico de caracteres (OCR), cujo objetivo é converter texto manuscrito ou impresso em caracteres digitais, os filtros ajudam a remover texturas de fundo ou ruídos do papel antes que o texto seja segmentado e interpretado. Em veículos autônomos, a filtragem em tempo real permite que os sistemas percebam melhor as marcações de faixa e detectem pedestres — mesmo sob condições de iluminação desafiadoras.

A filtragem também desempenha um papel fundamental no Aprendizado Profundo. Embora as Redes Neurais Convolucionais (CNNs) aprendam filtros automaticamente durante o treinamento, a ideia matemática por trás disso é a mesma: extrair características significativas a partir de dados brutos de pixels. De fato, as primeiras camadas das CNNs frequentemente se assemelham a detectores de bordas e filtros de desfoque tradicionais, reforçando a ideia de que a filtragem é um bloco de construção essencial da Visão Computacional, seja desenvolvida manualmente ou aprendida a partir dos dados.

Em resumo, a filtragem de imagens é o ponto de partida para traduzirmos pixels brutos em características significativas. Ao aplicar filtros cuidadosamente escolhidos — seja para suavizar, realçar, aumentar a nitidez ou detectar padrões —, equipamos nossos sistemas de visão computacional com as ferramentas necessárias para interpretar informações visuais.

Nas próximas seções, vamos explorar a base matemática da filtragem, analisar diferentes tipos de filtros e entender como essas técnicas são utilizadas em aplicações do mundo real.

Dica: para um aprofundamento teórico, consulte Gonzalez & Woods (2018), "*Digital Image Processing*", que oferece uma base abrangente sobre filtragem espacial; ou *Szeliski* (2022), "*Computer Vision: Algorithms and Applications*", que conecta a filtragem aos pipelines práticos de Visão Computacional.

FIAP

2 FUNDAMENTOS MATEMÁTICOS

Antes de compreendermos completamente como funcionam os filtros de imagem, precisamos entender as operações matemáticas que os sustentam. No coração da maioria das técnicas de filtragem está um conceito simples, porém poderoso: a **convolução**. A convolução oferece um mecanismo formal para aplicar uma pequena matriz (chamada de **kernel** ou filtro) a uma imagem com o objetivo de enfatizar ou suprimir certos padrões.

Matematicamente, a imagem é representada como uma matriz bidimensional de valores de intensidade, também chamada de matriz ou tensor. Uma imagem em tons de cinza é, geralmente, uma matriz 2D, enquanto uma imagem colorida é uma matriz 3D, com matrizes separadas para os canais vermelho, verde e azul. Já o kernel é uma matriz bem menor (por exemplo, 3×3 ou 5×5), que contém pesos utilizados para transformar os valores dos pixels da imagem.

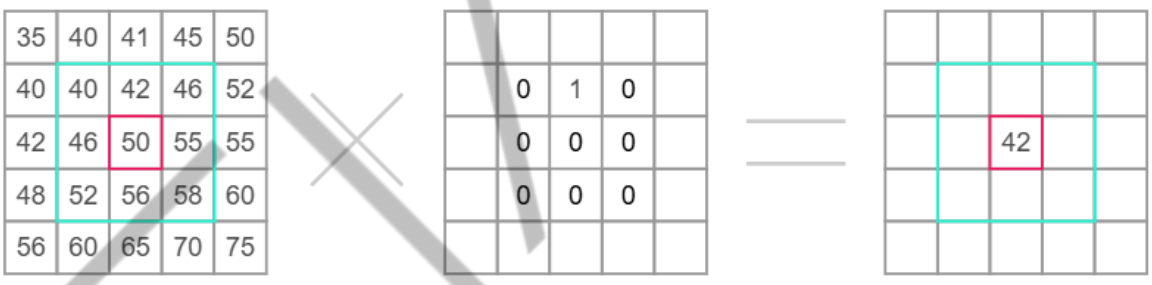


Figura 4 - Matriz de imagem em tons de cinza 5×5 e matriz de kernel 3×3
Fonte: CapivaraRex (2016), adaptado por FIAP (2026)

2.1 Operação de convolução

A convolução é uma operação matemática que combina duas funções para produzir uma terceira, expressando como o formato de uma é modificado pela outra. Na filtragem de imagens, convoluímos uma imagem $I(x, y)$ com um kernel $K(i, j)$ para produzir uma imagem filtrada $R(x, y)$, normalmente definida como:

$$R(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} k(\mathrm{i},j) * I(x-\mathrm{i}, y-j)$$

Na prática, isso significa posicionar o kernel sobre a imagem (centralizado em um pixel específico), multiplicar os valores sobrepostos, somá-los e armazenar o resultado em uma nova matriz na posição correspondente.

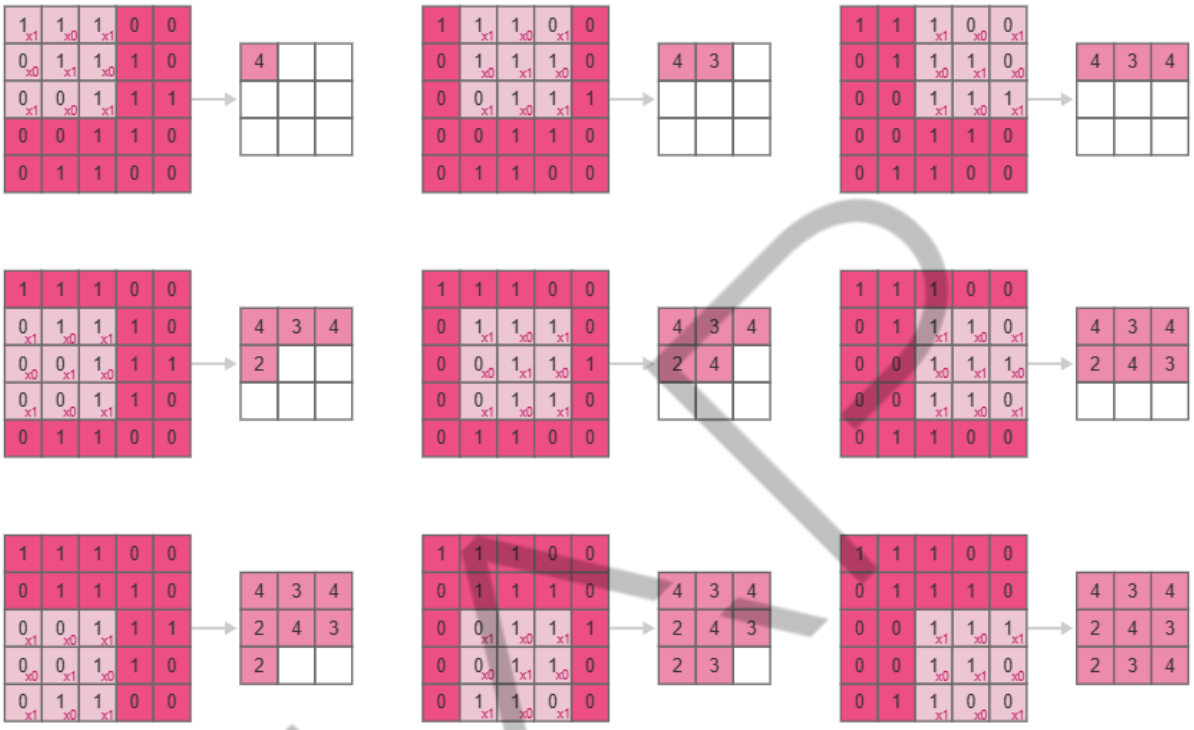


Figura 5 - Ilustração passo a passo do kernel 3×3 deslizando sobre uma imagem 5×5, mostrando multiplicação e soma
Fonte: EIALAB (2024), adaptado por FIAP (2026)

Esse processo é repetido para cada pixel da imagem, razão pela qual é chamado de operação de janela deslizante. É importante observar que, na definição matemática da convolução, o kernel é invertido horizontal e verticalmente antes da aplicação. Entretanto, na maioria das bibliotecas de processamento de imagem (como OpenCV ou SciPy), a operação implementada é, na verdade, correlação cruzada (*cross-correlation*), que omite a inversão. Apesar dessa distinção técnica, o termo “convolução” é amplamente utilizado na comunidade de Visão Computacional.

2.1.1 Padding e efeitos de borda

Um problema imediato da convolução ocorre nas bordas da imagem. Como o kernel exige pixels vizinhos para seus cálculos, é necessário decidir como lidar com as extremidades. Isso é resolvido com o uso de **padding** — a adição de linhas e

colunas extras ao redor da imagem. O padding pode ser composto por zeros (bordas pretas), pelo reflexo das bordas da imagem ou pela replicação do pixel mais próximo.

Sem o padding, a imagem de saída será menor do que a imagem original. Por exemplo, aplicar um filtro 3×3 a uma imagem 5×5, sem aplicar o padding, irá resultar em uma saída 3×3. O padding garante que a saída mantenha o tamanho original da imagem.

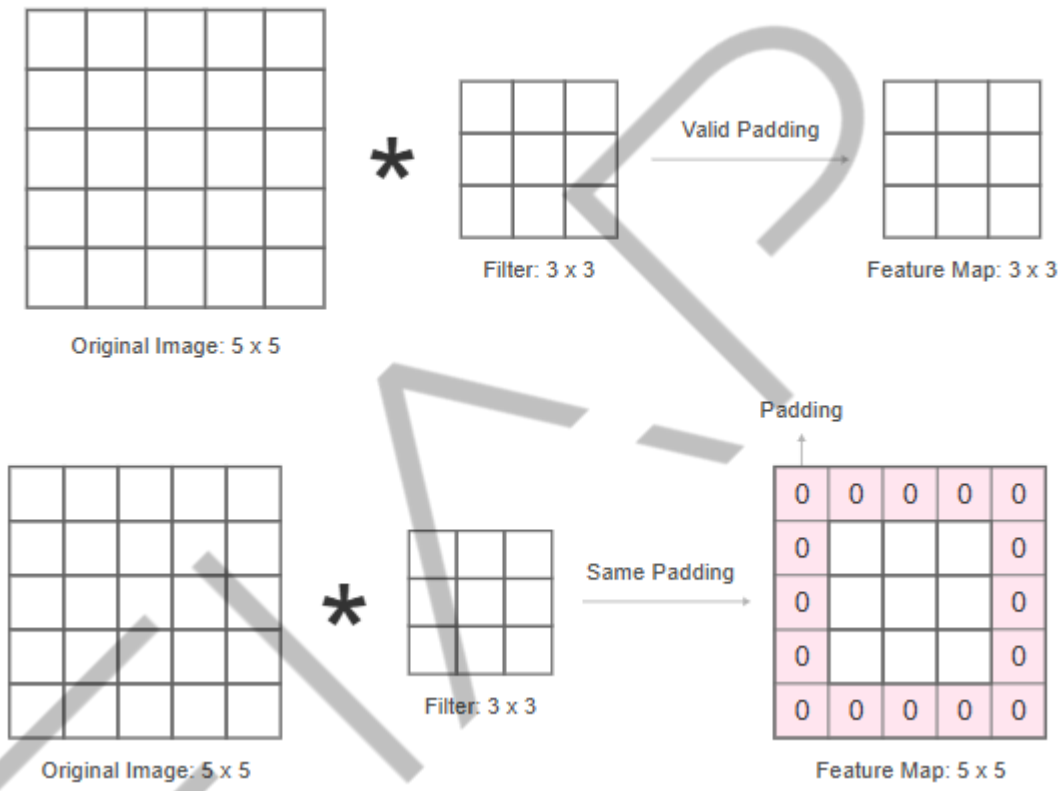


Figura 6 - Comparação visual entre convolução ‘válida’ (sem padding) e ‘mesma’ (com padding de zeros)
Fonte: Medium (2023), adaptado por FIAP (2025)

2.1.2 Stride e redução de resolução

O **stride** de um filtro determina a distância que o kernel se desloca na imagem. Um stride de 1 significa que o kernel se move um pixel por vez; um stride de 2 pula um pixel a cada passo. Strides maiores resultam em uma imagem de saída menor e podem ser utilizados para redução de resolução (*downsampling*).

Embora seja comum em arquiteturas de Aprendizado Profundo (como as CNNs), o uso de stride também pode ser útil em pipelines de visão clássica, como na detecção de objetos.

2.2 Linearidade e invariância a translação

A maioria dos filtros de imagem clássicos é linear, o que significa que o resultado de aplicar o filtro a uma combinação linear de imagens é igual à combinação dos resultados de cada imagem individualmente. A linearidade permite previsibilidade matemática e simplifica a análise.

Além disso, as operações de filtragem são geralmente invariantes a translações, ou seja, o resultado não muda se a imagem de entrada for deslocada no espaço. Essa propriedade é essencial em aplicações como reconhecimento de padrões, onde a posição dos elementos na imagem não deve afetar sua detecção.

2.2.1 Filtros separáveis

Alguns filtros, especialmente o filtro Gaussiano, são separáveis — isto é, a convolução 2D pode ser decomposta em duas convoluções 1D: uma horizontal e outra vertical. Isso reduz significativamente o custo computacional de $O(n^2)$ para $O(2n)$ por pixel.

Por exemplo, um filtro Gaussiano 5×5 pode ser implementado como um filtro 1×5 seguido por um filtro 5×1. Isso acelera o processo e mantém a precisão. Filtros separáveis são amplamente utilizados em sistemas em tempo real, como processamento de vídeo e visão embarcada.

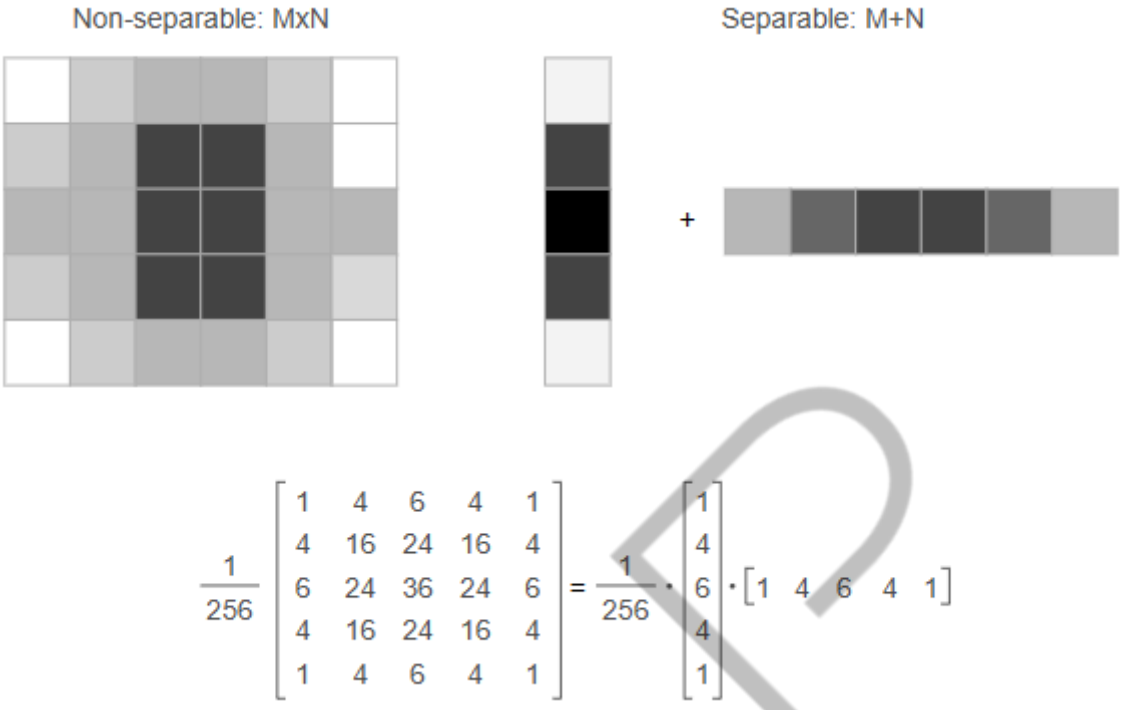


Figura 7 - O filtro Gaussiano como duas convoluções 1D consecutivas
Fonte: Medium (2020), adaptado por FIAP (2025)

2.3 Correlação *versus* convolução

Como mencionado anteriormente, a maioria das bibliotecas implementa a correlação, e não a convolução verdadeira. Embora estruturalmente semelhantes, a correlação não inverte o kernel antes de aplicá-lo. Essa distinção só se torna crítica quando o kernel não é simétrico. Por exemplo, os filtros de detecção de bordas, como Sobel e Prewitt, são assimétricos e produzirão resultados diferentes quando aplicados com convolução ou correlação.

Na prática, convolução e correlação produzem resultados muito semelhantes para kernels simétricos (como filtros de desfoque), e a performance e facilidade de implementação geralmente têm mais peso do que a aderência estrita à definição matemática.

Esses princípios matemáticos formam a base dos filtros de imagem. Compreender como a convolução funciona, junto com o papel do padding, stride e as propriedades do kernel, prepara o terreno para a construção de operações de filtragem mais complexas e poderosas. Nas próximas seções, exploraremos tipos

específicos de filtros, começando com os filtros lineares, como desfoque e nitidez, e depois avançando para técnicas mais sofisticadas e adaptativas.

Parâmetro	Descrição	Exemplos/valores comuns	Impacto no resultado
Tamanho do kernel	Define a largura e altura da janela (filtro) que percorre a imagem.	3x3, 5x5, 7x7	Filtros maiores capturam padrões mais globais; filtros menores focam em detalhes locais.
Stride	Número de pixels que o filtro “anda” ao se mover pela imagem (horizontal e verticalmente).	1, 2	Stride maior reduz o tamanho da saída (*downsampling*).
Padding	Adiciona pixels (geralmente zeros) nas bordas da imagem para controlar o tamanho da saída.	valid (sem padding), same (mantém tamanho)	Evita perda de informação nas bordas; same preserva dimensões.
Separabilidade	Refere-se à decomposição de uma convolução 2D em duas convoluções 1D (horizontal + vertical) ou convoluções por canais e espacial (*depthwise separable*).	Sim / Não (convoluções separáveis ou não)	Reduz significativamente o custo computacional sem grande perda de desempenho.

Quadro 1 - Resumo dos principais parâmetros de convolução — tamanho do kernel, stride, padding e separabilidade
Fonte: Elaborado pelo autor (2025)

3 FILTROS LINEARES

A filtragem linear é uma das técnicas mais fundamentais no processamento de imagens. Um filtro é considerado linear quando o valor do pixel de saída é uma combinação linear dos valores dos pixels de entrada. Isso significa que o filtro aplica uma soma ponderada a uma vizinhança de pixels, onde os pesos são definidos pelo kernel. Filtros lineares são populares por sua simplicidade, previsibilidade matemática e eficácia em várias tarefas centrais da Visão Computacional.

Nesta seção, exploraremos duas grandes categorias de filtros lineares: **filtros de suavização** — utilizados para reduzir ruído e borrar detalhes — e **filtros de nitidez** — usados para realçar bordas e estruturas finas. Em seguida, também examinaremos os filtros de **detecção de bordas**, que ficam entre suavização e nitidez, extraindo características relevantes enquanto suprimem o fundo irrelevante.

3.1 Filtros de suavização (passa-baixa)

Filtros de suavização, também conhecidos como filtros passa-baixa, atuam suprimindo os componentes de alta frequência da imagem. Componentes de alta frequência correspondem a mudanças rápidas de intensidade, como ruído, texturas finas ou objetos pequenos. Ao reduzir esses componentes, os filtros de suavização ajudam a tornar a imagem menos “poluída”, reduzindo detalhes, mas preservando formas e gradientes gerais.

O filtro de suavização mais básico é o **filtro da média** (ou *box filter*). Esse filtro substitui cada pixel pela média da sua vizinhança imediata.

Por exemplo, um filtro de média 3×3 é:

$$\kappa = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Esse kernel é convoluído com a imagem, e cada pixel de saída é a média de si mesmo com seus 8 vizinhos imediatos.

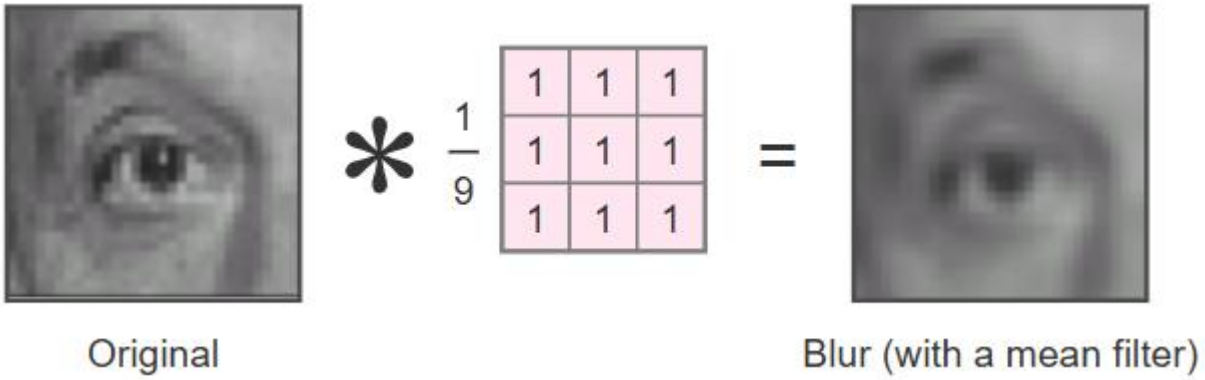


Figura 8 - Foto original *vs* resultado da aplicação do filtro de média 3×3
Fonte: Medium (2021), adaptado por FIAP (2025)

Embora o filtro da média seja computacionalmente barato, ele tem uma importante desvantagem: tende a borrar as bordas junto com o ruído. Em aplicações que exigem preservação da estrutura, isso pode ser um grande problema.

Uma abordagem mais refinada é o **filtro Gaussiano**, que utiliza uma média ponderada onde os pixels centrais contribuem mais do que os distantes. O kernel Gaussiano é definido pela função Gaussiana bidimensional:

$$G(x,y) = \frac{1}{2\pi\sigma^2} \mathrm{e}^{-\frac{x^2+y^2}{2\sigma^2}}$$

Aqui, **σ** controla a “dispersão” do desfoque: quanto maior o sigma, mais suavização é aplicada.

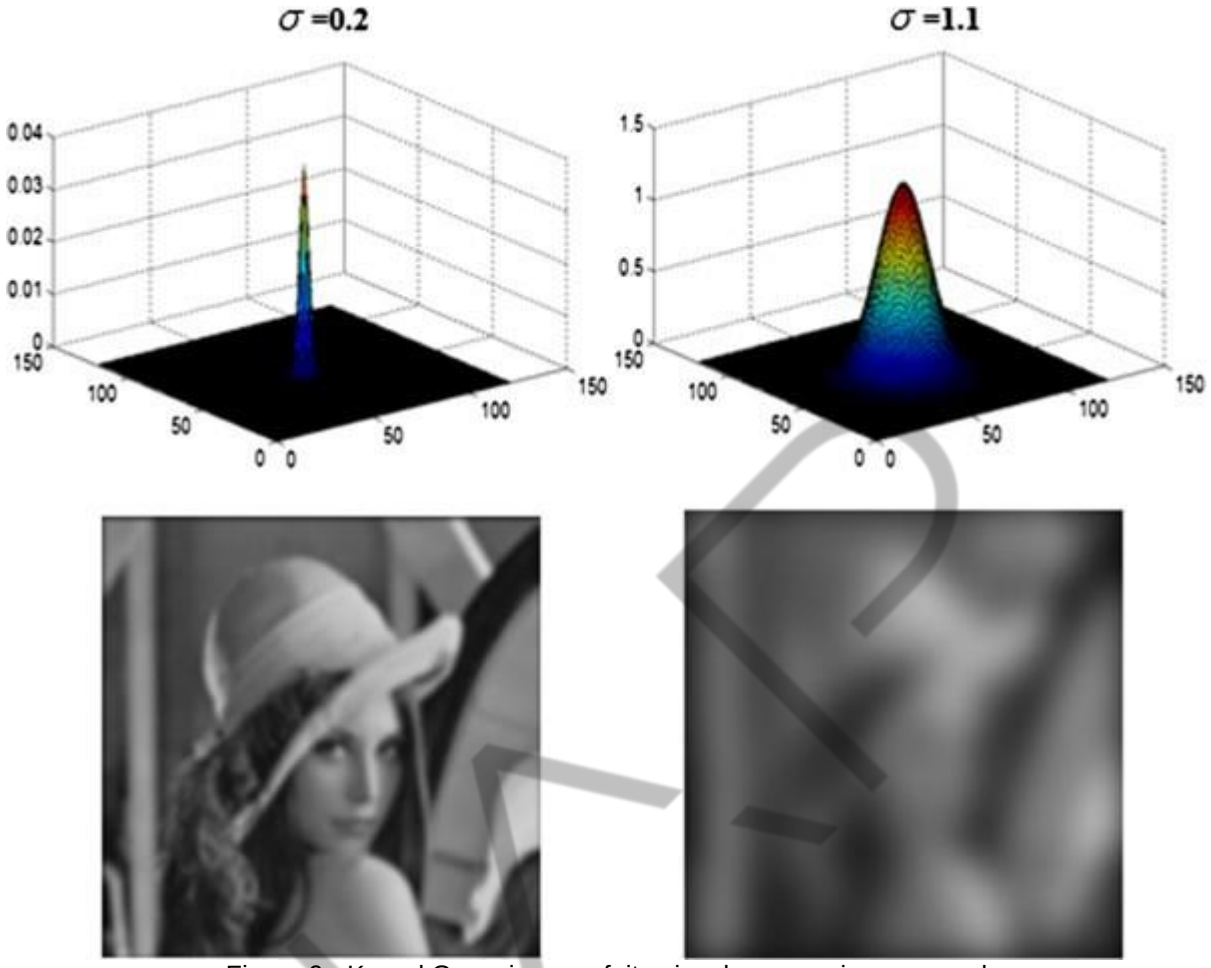


Figura 9 - Kernel Gaussiano e efeito visual em uma imagem real
Fonte: ResearchGate (2014)

A filtragem Gaussiana é amplamente utilizada para tarefas como remoção de ruído, pirâmides de imagem e pré-processamento com preservação de bordas. De fato, ela costuma ser o primeiro passo antes da aplicação de algoritmos mais complexos, como a detecção de objetos, pois simplifica a imagem ao suprimir detalhes irrelevantes.

3.2 Filtros de nitidez (passa-alta)

Enquanto os filtros de suavização removem detalhes, os filtros de nitidez fazem o oposto: enfatizam transições, como bordas e cantos, ao amplificar componentes de alta frequência da imagem. Esses filtros são particularmente úteis para realçar imagens borradas ou com pouco contraste.

Um dos métodos mais simples de nitidez é o ***unsharp masking***. Apesar do nome confuso, ele funciona subtraindo uma versão borrada da imagem da original e, em seguida, adicionando a diferença de volta para realçar as bordas:

$$I_sharp = I_original + \alpha(I_original - I_blurred)$$

O termo **α** controla a intensidade da nitidez aplicada.

Outro kernel de nitidez comum é o **filtro Laplaciano**, que destaca áreas de mudança rápida de intensidade. Ele se baseia na segunda derivada da imagem e tem a forma:

$$k = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Esse filtro é sensível às estruturas finas e é frequentemente utilizado para o realce geral de bordas.



Figura 10 - Exemplo de imagem antes e depois da filtragem Laplaciana
Fonte: Medium (2025)

No entanto, os filtros de nitidez são muito sensíveis ao ruído. Se a imagem estiver ruidosa, a nitidez amplificará tanto as bordas reais quanto o ruído. Por isso, é comum aplicar uma etapa de suavização (como o blur Gaussiano) antes da nitidez, especialmente em pipelines nos quais a qualidade da imagem é variável.

3.3 Filtros de detecção de bordas

A detecção de bordas talvez seja a forma mais icônica da filtragem linear. Detectar bordas equivale a identificar regiões da imagem onde a intensidade dos pixels muda abruptamente. Essas transições geralmente correspondem aos contornos dos objetos — cruciais para tarefas como segmentação, reconhecimento e rastreamento.

Um dos filtros de detecção de bordas mais utilizados é o **operador de Sobel**. Ele consiste em dois kernels 3×3: um para detectar bordas horizontais e outro para verticais.

- **Horizontal (direção x):**

$$k_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- **Vertical (direção y):**

$$k_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Aplicando ambos os kernels, obtemos dois mapas de gradiente, $\mathbf{G_x}$ e $\mathbf{G_y}$, a partir dos quais podemos calcular:

- **Magnitude do gradiente:**

$$G = \sqrt{G_x^2 + G_y^2}$$

- **Direção da borda:**

$$\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$$

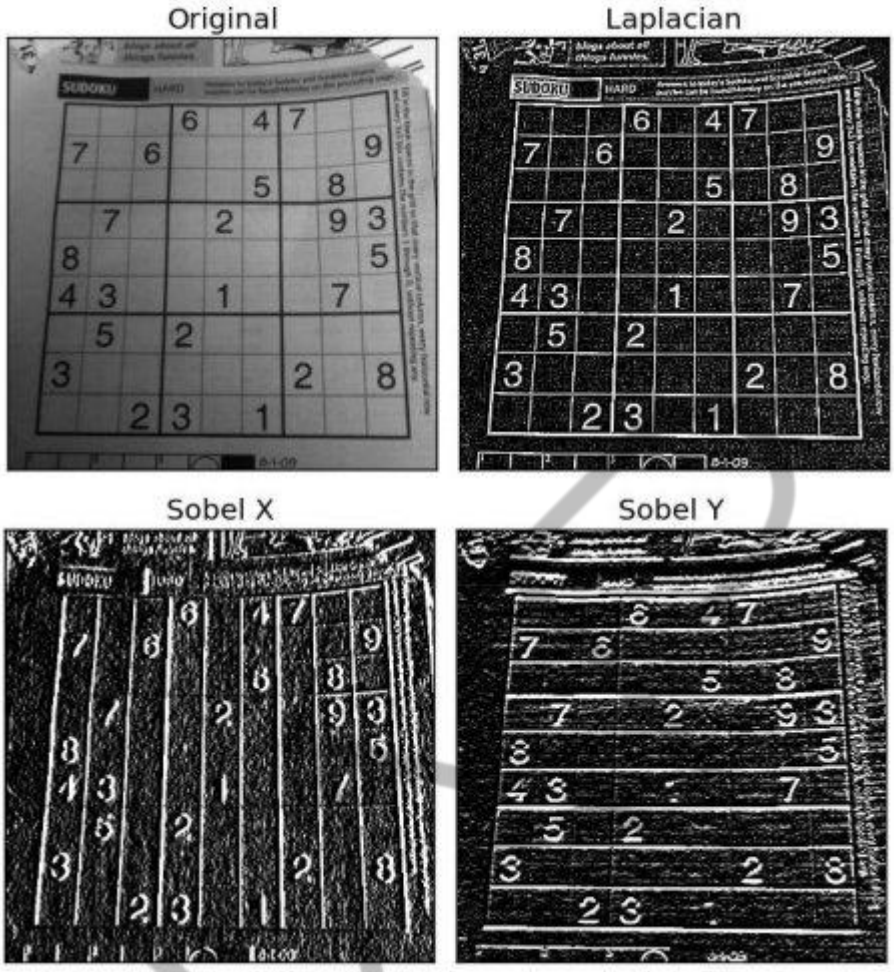


Figura 11 - Filtro Sobel
Fonte: OpenCV (2018)

Os **filtros Sobel** são robustos a pequenas variações e produzem resultados mais suaves do que operadores de diferença simples. Eles são amplamente utilizados em tarefas clássicas de Visão Computacional e continuam relevantes até hoje, muitas vezes como ferramenta de pré-processamento ou visualização.

O **filtro de Prewitt** é uma variante ligeiramente mais simples do Sobel, com pesos uniformes, enquanto o **operador de Roberts** é um filtro ainda menor, que utiliza kernels 2×2 para detectar bordas diagonais. Todos esses operadores são exemplos de filtros passa-alta, projetados para amplificar transições espaciais de intensidade.

3.4 Comparação e casos de uso

Cada filtro linear possui vantagens e desvantagens. Filtros de média são rápidos, mas ruins na preservação de estrutura. Filtros Gaussianos mantêm melhor as bordas, embora tenham um custo computacional mais elevado. Filtros Laplacianos

realçam detalhes, porém são sensíveis ao ruído. Já os filtros de Sobel e Prewitt são eficazes na extração de estruturas, mas podem precisar de pós-processamento para limpar as bordas.

A escolha do filtro ideal depende da aplicação. Em digitalização de documentos, por exemplo, a suavização Gaussiana seguida de detecção de bordas auxilia no isolamento de traços manuscritos. Em navegação autônoma, filtros Sobel podem identificar faixas na estrada. Em imagens de satélite, o *unsharp masking* realça feições do terreno, auxiliando em análises geoespaciais.



Figura 12 - Vários tipos de filtros podem ser desenvolvidos e aplicados a imagem
Fonte: IEEE Xplore (2022)

Como podemos observar, os filtros lineares são ferramentas versáteis que formam os blocos de construção de muitos pipelines de Visão Computacional. Na próxima seção, mudaremos o foco para os **filtros não lineares**, que lidam com modelos de ruído mais complexos e apresentam melhor desempenho em situações onde as suposições lineares falham.

4 FILTROS NÃO LINEARES

Embora os filtros lineares sejam poderosos e amplamente utilizados, eles possuem limitações importantes, especialmente quando se trata de lidar com ruído e preservar bordas ao mesmo tempo. É nesse contexto que entram os **filtros não lineares**. Como o próprio nome indica, esses filtros não se baseiam em combinações lineares dos valores dos pixels. Em vez disso, aplicam funções não lineares sobre vizinhanças locais, o que lhes confere uma vantagem significativa em tarefas específicas, como a remoção de ruído do tipo impulso (*salt-and-pepper*), a suavização com preservação de bordas e a filtragem baseada em mediana.

Os filtros não lineares são particularmente eficazes em cenários do mundo real, nos quais o ruído não segue uma distribuição Gaussiana ou quando a preservação de bordas nítidas é essencial. Neste tópico, exploraremos alguns dos filtros não lineares mais comuns: **filtros de mediana**, **filtros bilaterais** e **filtros guiados**.

4.1 Filtro de mediana

O filtro de mediana é um dos filtros não lineares mais simples e eficazes. Em vez de calcular a média dos pixels vizinhos (como faz o filtro de média), o filtro de mediana substitui o pixel central de uma vizinhança pelo valor mediano dos pixels ao redor.

Isso torna o filtro de mediana particularmente eficaz contra o **ruído do "tipo sal e pimenta"**, que aparece como pixels pretos e brancos aleatórios espalhados pela imagem.

Para aplicar um filtro de mediana:

1. Defina uma vizinhança (tipicamente 3×3 ou 5×5).
2. Colete todos os valores de pixels dentro dessa janela.
3. Ordene os valores e selecione a mediana.
4. Atribua esse valor mediano ao pixel central.

Por exemplo, se uma vizinhança 3×3 contém os valores [3, 5, 7, 9, 2, 6, 1, 4, 8], a lista ordenada é [1, 2, 3, 4, 5, 6, 7, 8, 9] e a mediana é 5.

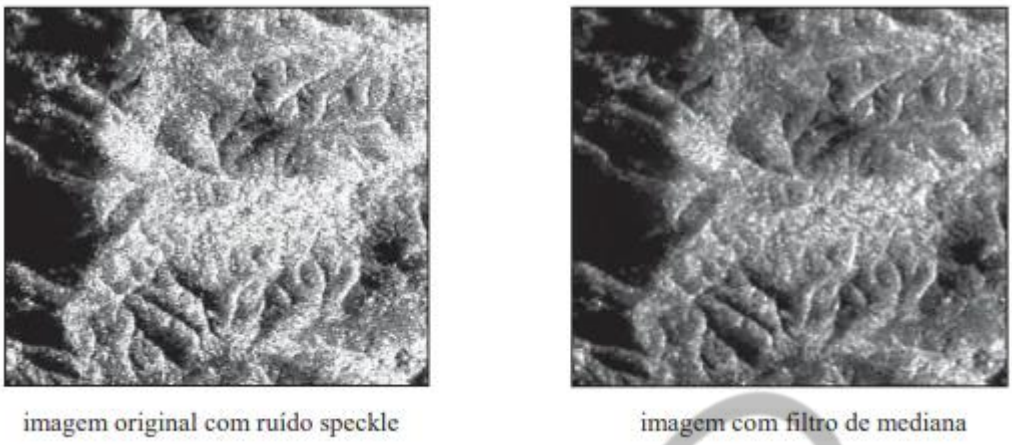


Figura 13 - Filtro da mediana
Fonte: Adenilson Giovanini (2025)

Diferente dos filtros lineares, o filtro de mediana **preserva as bordas**, pois não realiza a média entre regiões com transições. No entanto, ele pode distorcer levemente texturas finas ou padrões repetidos, especialmente quando a janela é muito grande.

Filtros de mediana são amplamente utilizados em **imagens médicas**, **escaneamento de documentos** e **visão robótica**, pois removem pixels fora do padrão sem distorcer bordas, o que é essencial nesses contextos.

4.2 Filtro bilateral

O **filtro bilateral** é uma técnica não linear mais avançada, projetada para realizar suavização com preservação de bordas. Proposto por Tomasi e Manduchi em 1998, ele se tornou um padrão tanto na Visão Computacional clássica quanto em sistemas em tempo real.

O que torna o filtro bilateral único é o fato de ele considerar **tanto a distância espacial entre os pixels** quanto **à similaridade de intensidade** ao calcular a média ponderada. Ou seja, pixels que estão próximos e têm valores de intensidade parecidos recebem maior peso, enquanto pixels distantes ou muito diferentes contribuem menos para o resultado.

Matematicamente, para um pixel p, o filtro bilateral calcula:

$$I_{filtered}(P) = \frac{1}{w_p} \sum_{qE\Omega} I(q) f_s(\|p - q\|) f_r(|I(p) - I(q)|)$$

Em que:

- f_s: função Gaussiana espacial (proximidade entre pixels);
- f_r: função Gaussiana de intensidade (semelhança de brilho);
- w_p: fator de normalização.



Figura 14 - Filtro da bilateral
Fonte: AI Learning Centre (2017)

O resultado é que o filtro bilateral suaviza regiões com variações graduais de intensidade (como céu ou paredes), mas **preserva bordas**, já que descontinuidades de intensidade recebem peso muito baixo.

Essa propriedade torna o filtro bilateral ideal para:

- Remoção de ruído;
- Efeitos de cartoonização;
- Pré-processamento em sistemas de detecção facial;
- Mapeamento de tons HDR;
- **Suavização de vídeo** com preservação de contornos.

Contudo, o filtro bilateral é **computacionalmente caro**, especialmente para imagens grandes, pois cada pixel exige cálculos que consideram a distância espacial e de intensidade. Implementações modernas utilizam aproximações ou aceleração via GPU para atingir desempenho em tempo real.

4.3 Filtro guiado

O **filtro guiado** é uma técnica relativamente nova, introduzida em 2010 por He, Sun e Tang. Ele resolve algumas limitações do filtro bilateral, oferecendo um desempenho mais rápido e melhorando o comportamento em bordas fortes.

Esse filtro utiliza uma **imagem guia**, que pode ser a própria imagem de entrada ou outra imagem, para orientar o processo de suavização. Em essência, ele ajusta um modelo linear local entre a imagem guia e a imagem de saída, garantindo que o filtro respeite a estrutura da imagem de referência. Esse modelo se adapta com base em estatísticas locais (como média e variância), e sua saída é garantidamente preservadora de bordas.

O filtro guiado é utilizado em:

- Edição de fotos (esmaecimento e matting);
- Refinamento de mapas de profundidade em reconstrução 3D;
- Suavização com preservação de bordas na fotografia computacional.

Uma grande vantagem do filtro guiado é sua **eficiência computacional**, especialmente quando implementado com imagens integrais, o que permite processar imagens grandes rapidamente.

FIAP

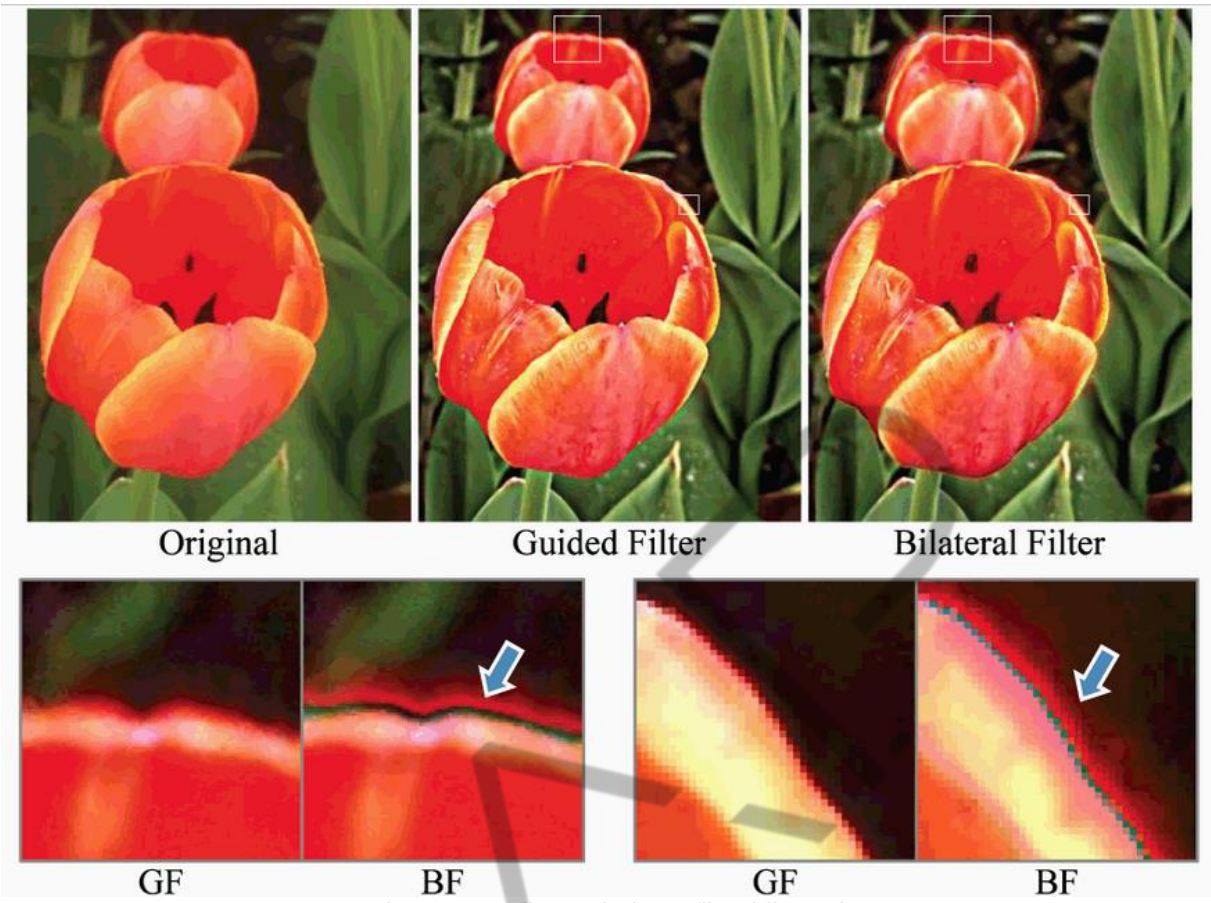


Figura 15 - Filtro guiado *vs* filtro bilateral
Fonte: IEEE Computer Society (2013)

4.4 Filtragem não linear: pontos fortes e limitações

Os filtros não lineares se destacam em tarefas nas quais os filtros lineares falham. Por exemplo, considere uma câmera de segurança afetada por ruído do tipo “sal e pimenta” devido à compressão do vídeo. Um filtro de mediana pode remover esse ruído quase sem impactar a estrutura da cena. Da mesma forma, em aplicativos de retoque de retratos, filtros bilaterais ou guiados eliminam imperfeições e inconsistências de textura enquanto mantêm contornos nítidos, como olhos e lábios.

No entanto, filtros não lineares também têm alguns desafios. Eles costumam ser **mais lentos** que os filtros lineares, **mais difíceis de configurar**, e podem ter **comportamento imprevisível** quando aplicados a imagens sintéticas ou com muitas variações de alta frequência. O uso excessivo pode resultar em um aspecto muito “plástico”, especialmente em fotografias.

4.5 Considerações finais

Os filtros não lineares são ferramentas poderosas para suavização e remoção de ruído, com o grande benefício de preservar bordas. Enquanto os filtros de mediana são simples e eficazes contra ruído impulsivo, os filtros bilateral e guiado oferecem suavização avançada adequada para sistemas de visão de alta qualidade e em tempo real.

Na próxima seção, exploraremos como esses conceitos de filtragem são aplicados no **domínio da frequência**, no qual transformações como a **Transformada de Fourier** permitem manipular imagens com base em seu conteúdo espectral.

FIAP

5 FILTROS NO DOMÍNIO DA FREQUÊNCIA

Até o momento, trabalhamos com filtragem de imagens no **domínio espacial**, ou seja, manipulando diretamente as intensidades dos pixels utilizando operações de vizinhança, como a convolução. No entanto, existe uma abordagem totalmente diferente, e poderosa, de filtrar imagens: **analisando e manipulando seus componentes de frequência**. Essa técnica, conhecida como **filtragem no domínio da frequência**, usa ferramentas como a **Transformada de Fourier** para revelar e processar a estrutura da imagem em termos de padrões periódicos, em vez de intensidades de pixel.

Essa técnica tem base na teoria de processamento de sinais e possui muitas aplicações em Visão Computacional, compressão de imagens e até em aprendizado profundo. Compreender a filtragem no domínio da frequência oferece uma visão profunda de como a informação é distribuída em uma imagem e como podemos manipular estruturas específicas (como texturas, padrões ou ruído) de forma mais fundamentada.

5.1 O que é o domínio da frequência?

O domínio da frequência representa uma imagem em termos de **componentes senoidais**, ondas com diferentes frequências e orientações. Toda imagem, por mais complexa que seja, pode ser decomposta em uma soma de ondas senoidais usando a **Transformada de Fourier**. Esses componentes descrevem como os valores de intensidade variam espacialmente na imagem.

Em termos simples:

- **Frequências baixas** representam variações lentas de intensidade — como fundos suaves ou grandes áreas de cor.
- **Frequências altas** capturam mudanças rápidas — como bordas, texturas finas e ruído.

Ao transformar uma imagem para o domínio da frequência, podemos **remover ou realçar seletivamente diferentes faixas de frequência** e depois **transformá-la de volta** para o domínio espacial.

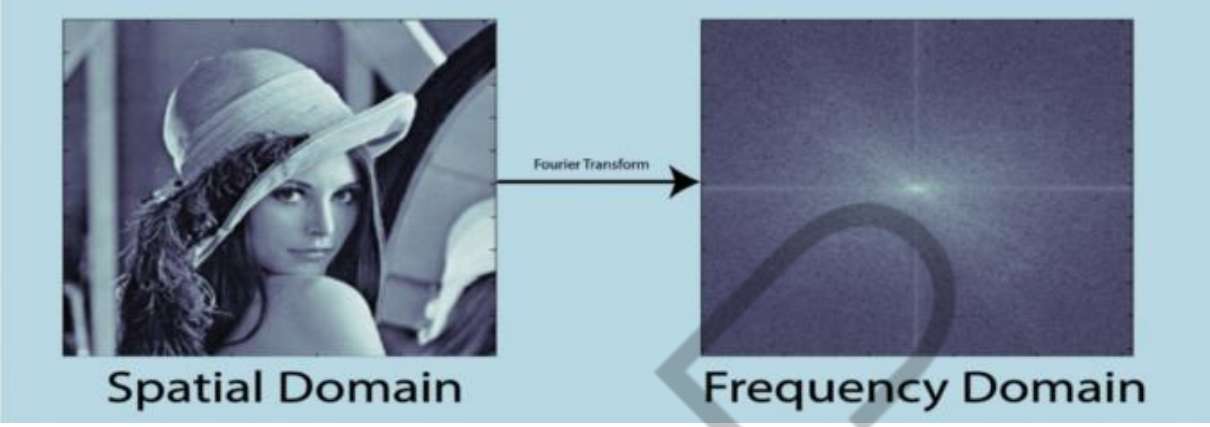


Figura 16 - Imagem no domínio espacial e no domínio da frequência
Fonte: Medium (2019)

5.2 Transformada de Fourier 2D

A **Transformada Discreta de Fourier (DFT)** bidimensional é definida como:

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \mathrm{e}^{-2\pi \mathrm{i}\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

Em que:

- $f(x,y)$ é a imagem original no domínio espacial;
- $F(u,v)$ é a representação da imagem no domínio da frequência;
- M e N são as dimensões da imagem, linhas e colunas;
- Cada ponto (u,v) representa uma frequência específica e sua orientação.

Na prática, utilizamos a **Transformada Rápida de Fourier (FFT)** para calcular essa transformação de forma eficiente.

Dica: está disponível em bibliotecas como **NumPy** (np.fft.fft2) e **OpenCV**.

Como a saída bruta da FFT é **complexa** e difícil de visualizar, normalmente plotamos o **espectro de magnitude** usando a **escala logarítmica**:

$$Magnitude = log(1 + |F(u, v)|)$$

Por padrão, a saída da FFT coloca o componente de **frequência zero** (DC) nos cantos da imagem. Ao utilizar funções como fftshift() para centralizar esse componente, facilitamos a análise.

5.3 Filtros passa-baixa e passa-alta

Assim como no domínio espacial, podemos criar **filtros passa-baixa** e **passa-alta** no domínio da frequência.

- **Filtro passa-baixa:** mantém os componentes de baixa frequência e remove os de alta frequência. Isso suaviza a imagem, eliminando transições abruptas.
- **Filtro passa-alta:** preserva as altas frequências e remove as baixas, destacando bordas e texturas.

Isso é feito multiplicando a representação de frequência $F(u, v)$ por uma **máscara de filtro** $H(u, v)$:

$$F'(u, v) = F(u, v)H(u, v)$$

Em seguida, aplicamos a transformada inversa de Fourier para voltar ao domínio espacial:

$$f'(x, y) = IFFT2 \ \ \left(F'(u, v)\right)$$

- Exemplo de suavização via filtro passa-baixa:
  - Calcule a FFT da imagem;
  - Crie uma **máscara circular** que preserve apenas as frequências baixas;
  - Multiplique a FFT pela máscara;
  - Aplique a FFT inversa;
  - Exibe a **parte real** do resultado.

Esse método produz um resultado semelhante ao desfoque Gaussiano no domínio espacial, mas com **maior controle sobre o conteúdo de frequência**.

5.4 Filtragem: Domínio da frequência *versus* domínio espacial

Embora a filtragem no domínio da frequência pareça mais matemática, ela apresenta algumas vantagens importantes:

- **Operações globais:** os filtros atuam sobre toda a imagem de uma vez, úteis para ajustes amplos como remoção de ruído em regiões extensas;
- **Alta precisão:** podemos projetar filtros que atuam exatamente sobre faixas específicas de frequência — algo difícil no domínio espacial;
- **Eficiência com kernels grandes:** para convoluções com kernels muito grandes, é mais rápido filtrar no domínio da frequência, já que a convolução no espaço é equivalente à multiplicação no domínio da frequência.

Desvantagens:

- Requer transformações de ida e volta entre os domínios;
- Pode gerar **artefatos de borda** (exemplo: padding), caso o tratamento não seja adequado;
- Pode ter **baixa localização espacial** — isto é, pequenas estruturas podem se espalhar ou se desfocar.

5.5 Considerações finais

A filtragem no domínio da frequência abre uma **nova dimensão** para entender e manipular imagens. Enquanto a filtragem espacial é intuitiva e localizada, as operações no domínio da frequência oferecem uma abordagem **global** e **consciente do conteúdo espectral**, sendo ideais para suavização, nitidez, compressão e análise de padrões periódicos.

6 CASOS DE USO EM VISÃO COMPUTACIONAL E IMAGEM

Técnicas no domínio da frequência são poderosas em várias aplicações:

- **Compressão JPEG:** usa a **Transformada Cosseno Discreta (DCT)**, parente da Fourier, para remover informação de alta frequência que o olho humano ignora;
- **Imagem médica (como ressonância magnética, tomografia):** os dados são adquiridos diretamente no domínio da frequência e convertidos via transformadas inversas;
- **Sensoriamento remoto:** análise de frequência ajuda a detectar padrões periódicos em terrenos ou estruturas urbanas;
- **Reconhecimento de padrões:** descritores de Fourier analisam repetições e texturas com sensibilidade à frequência.

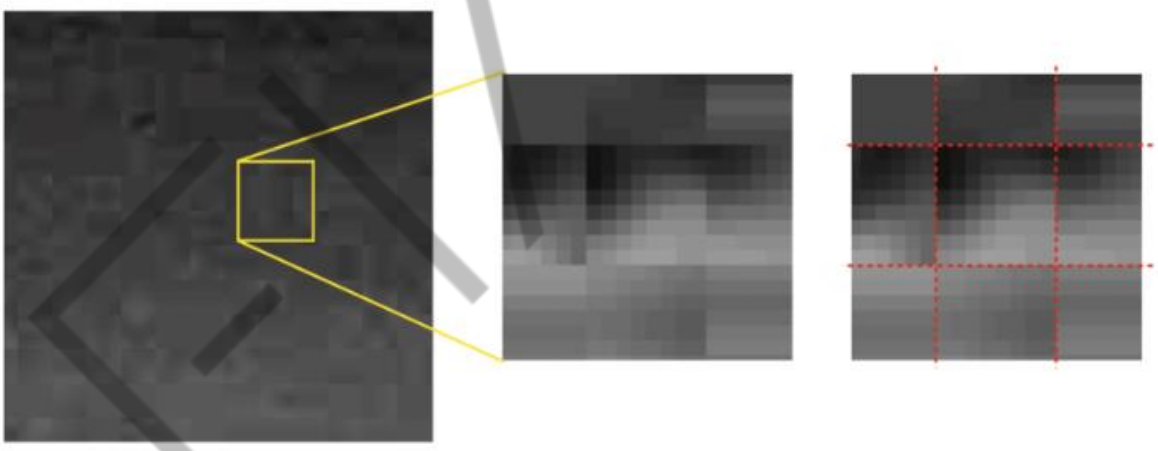


Figura 17 - Artefatos de bloco JPEG. As linhas pontilhadas vermelhas destacam os limites dos blocos 8 × 8 usados na compressão
Fonte: ResearchGate (2020)

No **aprendizado profundo**, a análise de frequência também ganha relevância, por exemplo, para estudar a **robustez de modelos** contra perturbações ou entender como as redes neurais processam textura *versus* forma.

A seguir, vamos explorar **como os filtros são implementados na prática**, tanto com operações de convolução em bibliotecas, como OpenCV e NumPy, quanto em frameworks de aprendizado de máquina, como TensorFlow e PyTorch.

7 LABORATÓRIO

7.1 Implementando filtros no NumPy

A seguir, apresentamos um exemplo básico de implementação de um filtro de média (uma média 3×3) com NumPy:

```
import numpy as np
from scipy.signal import convolve2d
import matplotlib.pyplot as plt
from skimage import data, color

image = color.rgb2gray(data.astronaut())  # Sample grayscale
image
kernel = np.ones((3, 3)) / 9  # 3x3 mean filter

blurred = convolve2d(image, kernel, mode='same',
boundary='symm')

plt.imshow(blurred, cmap='gray')
plt.title("Mean Filter (3x3)")
plt.axis('off')
plt.show()
```

Código-fonte 1 - Filtro da média com NumPy
Fonte: Elaborado pelo autor (2025)

Este exemplo utiliza o convolve2d do SciPy, que aplica o kernel usando convolução 2D com preenchimento simétrico. Você pode experimentar outros kernels, como Sobel, Gaussiano ou detectores de borda personalizados. O resultado do código pode ser visto na figura a seguir:



Figura 18 - Filtro da média
Fonte: Elaborado pelo autor (2025)

O OpenCV oferece funções otimizadas para operações de filtragem. Veja como aplicar um blur Gaussiano e um detector de bordas Laplaciano:

```
import cv2
import numpy as np
from scipy.signal import convolve2d
import matplotlib.pyplot as plt
from skimage import data, color

# Load the image
image = color.rgb2gray(data.astronaut())  # Sample
grayscale image

# Gaussian Blur
blurred = cv2.GaussianBlur(image, (5, 5), sigmaX=1.0)

# Laplacian for edge detection
edges = cv2.Laplacian(image, ddepth=cv2.CV_64F)


# display using matplotlib
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(blurred, cmap="gray")
plt.title("Blurred (Gaussian)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(
    edges, cmap="gray"
)  # Display edges, potentially normalize for better
visualization
plt.title("Edges (Laplacian)")
plt.axis("off")

plt.show()
```

Código-fonte 2 - Filtro Gaussiano e filtro Laplaciano
Fonte: Elaborado pelo autor (2025)

As funções do OpenCV são altamente otimizadas e usadas em sistemas de Visão Computacional em tempo real, como detecção e rastreamento de objetos. O código anterior gera como resultado as imagens filtradas com o filtro Gaussiano e com o filtro Laplaciano:

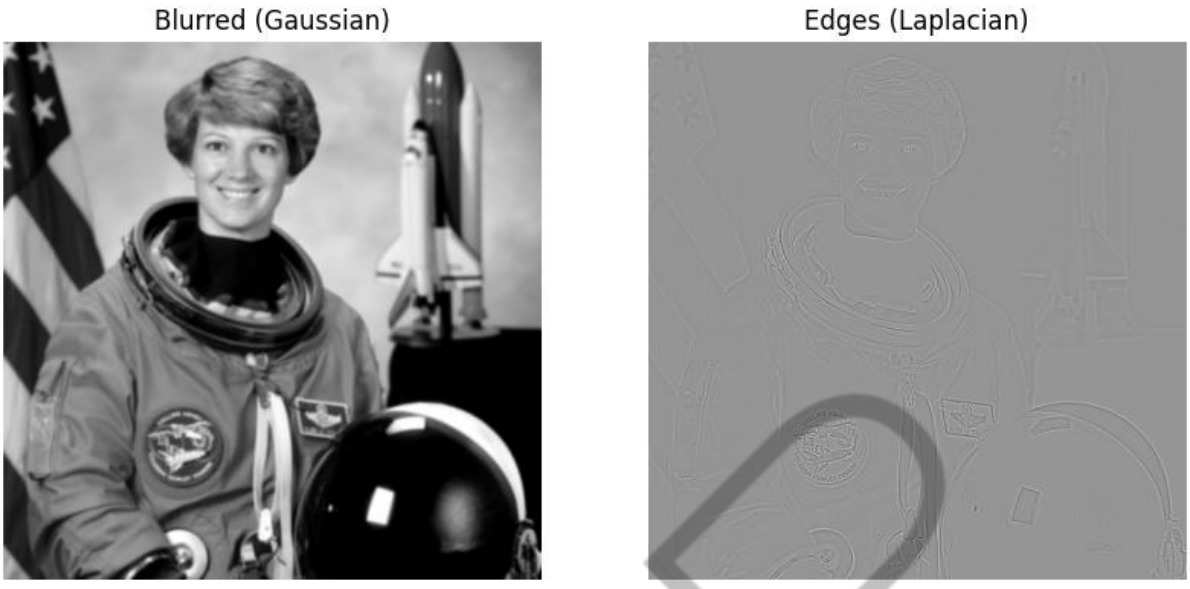


Figura 19 - Filtro Gaussiano e Laplaciano
Fonte: Elaborado pelo autor (2025)

7.2 Convolução em Deep Learning (com PyTorch)

Em aprendizado profundo, especialmente em redes neurais convolucionais, as camadas de convolução aprendem filtros de maneira automática a partir dos dados. No entanto, ainda podemos aplicar filtros estáticos manualmente usando o PyTorch para fins educacionais.

```
    import torch
    import torch.nn.functional as F
    import matplotlib.pyplot as plt
    from skimage import data, color
    import numpy as np

    # Load the image
    image = color.rgb2gray(data.astronaut())    # Sample
grayscale image

    # Edge detection kernel (Sobel X and Sobel Y)
    sobel_x = torch.tensor([[[-1, 0, 1], [-2, 0, 2], [-1,
0, 1]]], dtype=torch.float32)
    sobel_x = sobel_x.view(1, 1, 3, 3)

    sobel_y = torch.tensor([[[-1, -2, -1], [0, 0, 0], [1,
2, 1]]], dtype=torch.float32)
    sobel_y = sobel_y.view(1, 1, 3, 3)
```

```
    # Convert the numpy image to a PyTorch tensor
    # Add a batch and channel dimension
    image_tensor                                          =
torch.from_numpy(image).unsqueeze(0).unsqueeze(0).float()

    # Apply convolution for Sobel X and Sobel Y
    edge_x = F.conv2d(image_tensor, sobel_x, padding=1)
    edge_y = F.conv2d(image_tensor, sobel_y, padding=1)

    # Calculate the magnitude of the gradient
    # This gives a combined edge map
    edge_magnitude = torch.sqrt(edge_x**2 + edge_y**2)

    # Convert the output tensors back to numpy arrays for
displaying with matplotlib
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(edge_x.squeeze().detach().numpy(),
cmap="gray")
    plt.title("Sobel X Filtered Image")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(edge_y.squeeze().detach().numpy(),
cmap="gray")
    plt.title("Sobel Y Filtered Image")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(edge_magnitude.squeeze().detach().numpy(),
cmap="gray")
    plt.title("Sobel Edge Magnitude")
    plt.axis("off")

    plt.show()
```

Código-fonte 3 - Filtro de Sobel
Fonte: Elaborado pelo autor (2025)

O código anterior demonstra como criar e aplicar um filtro de Sobel com as funções do PyTorch, que operam de maneira similar a forma como uma camada CNN opera internamente. Na prática, as CNNs aprendem esses filtros durante o treinamento, o que permite extrair recursos significativos como bordas, texturas e partes de objetos. Esse processo será abordado nos próximos capítulos. O resultado do código pode ser visto na figura “Filtro de Sobel”.

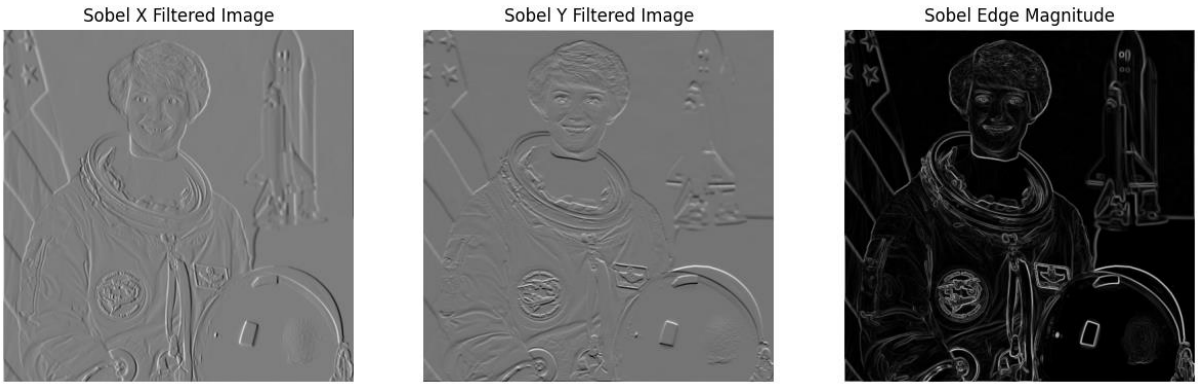


Figura 20 - Filtro de Sobel
Fonte: Elaborado pelo autor (2025)

7.3 Exemplo de filtro personalizado: nitidez

Vamos dar uma olhada em um filtro de nitidez personalizado, que aprimora bordas e detalhes:

```
    sharpen_kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])

    sharpened = convolve2d(image, sharpen_kernel, mode="same", boundary="symm")

    plt.imshow(sharpened, cmap="gray")
    plt.title("Sharpened Image")
    plt.axis("off")
    plt.show()
```

Código-fonte 4 - Filtro de nitidez
Fonte: Elaborado pelo autor (2025)

Esse tipo de filtro é útil em aplicações como aprimoramento de documentos, imagens forenses e preparação de entradas para OCR. O resultado está apresentado a seguir:



Figura 21 - Filtro de nitidez
Fonte: Elaborado pelo autor (2025)

8 EXEMPLOS PRÁTICOS E APLICAÇÕES

Agora que exploramos a teoria e a mecânica da filtragem de imagens, é hora de vê-la em ação. A filtragem não é apenas uma curiosidade técnica, é uma ferramenta fundamental usada em uma ampla gama de setores e disciplinas. Desde o aprimoramento de fotografias até a habilitação de carros autônomos, os filtros estão presentes em quase todas as etapas do processamento de dados visuais.

Nesta seção, abordaremos diversas aplicações reais da filtragem. Vamos examinar como diferentes filtros resolvem problemas específicos, quais compensações estão envolvidas e como interpretar seus efeitos em dados reais. Ao conectar a teoria à prática, entenderemos melhor como selecionar, ajustar e aplicar filtros de maneiras significativas.

8.1 Redução de ruído em fotografias ruidosas

Um dos problemas mais comuns no processamento de imagens é o ruído, principalmente em condições de pouca luz. As fotografias tiradas com ISO alto ou em dispositivos móveis em ambientes escuros frequentemente apresentam artefatos granulares, manchas aleatórias ou ruído “sal e pimenta”.

Para resolver isso, usamos filtros de suavização — geralmente filtros gaussianos ou medianas.

- **Exemplo:** uma imagem com ruído em tons de cinza de uma cena noturna de rua é processada usando:
  - Um filtro mediana 3×3.
  - Um filtro Gaussiano 5×5 com $\sigma = 1.2$.

```
    from skimage import data, util, filters
    from scipy.ndimage import gaussian_filter,
median_filter
    import matplotlib.pyplot as plt

    image = util.img_as_float(data.astronaut())
    noisy = util.random_noise(image, mode="s&p",
amount=0.05)

    median = median_filter(noisy, size=3)
    gaussian = gaussian_filter(noisy, sigma=1)

    # Plot
    fig, ax = plt.subplots(1, 3, figsize=(15, 5))
    ax[0].imshow(noisy)
    ax[0].set_title("Noisy Image")
    ax[1].imshow(median)
    ax[1].set_title("Median Filtered")
    ax[2].imshow(gaussian)
    ax[2].set_title("Gaussian Filtered")
    for a in ax:
        a.axis("off")
    plt.show()
```

Código-fonte 5 - Remoção de ruídos
Fonte: Elaborado pelo autor (2025)

O filtro da mediana remove com sucesso o ruído, preservando as bordas ao redor de edifícios e placas. O filtro Gaussiano produz um resultado ligeiramente mais desfocado, melhor na remoção de ruído Gaussiano aleatório, mas com alguma perda de detalhes.

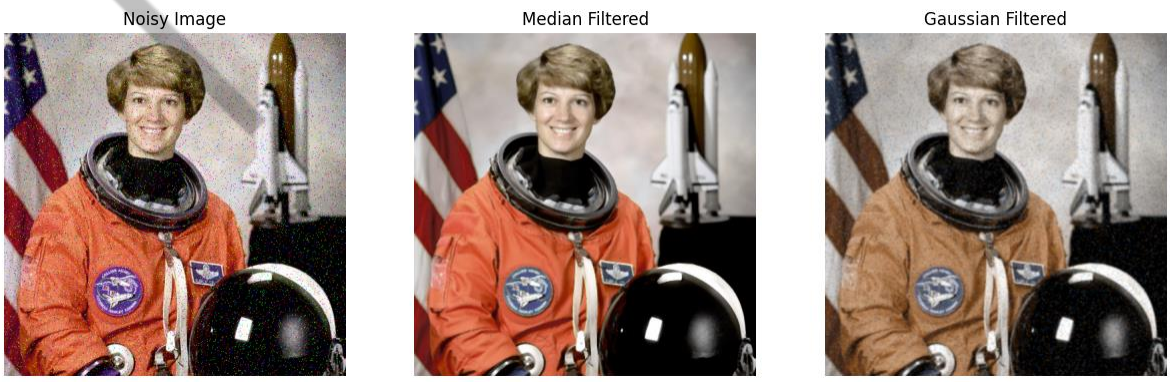


Figura 22 - Removendo ruídos de imagens
Fonte: Elaborado pelo autor (2025)

8.2 Aprimorando documentos para OCR com o filtro Sharpening

Para simular a melhoria da visibilidade em materiais digitalizados ou fotos de baixo contraste, aplicamos um filtro de nitidez para destacar bordas e melhorar a clareza dos detalhes.

```
    import numpy as np
    from scipy.signal import convolve2d

    sharpen_kernel = np.array([[0, -1, 0], [-1, 5, -1], [0,
-1, 0]])

    sharpened = np.zeros_like(image)
    for c in range(3):
        sharpened[..., c] = convolve2d(
            image[...,  c],  sharpen_kernel,  mode="same",
boundary="symm"
        )

    fig, ax = plt.subplots(1, 2, figsize=(12, 5))
    ax[0].imshow(image)
    ax[0].set_title("Original")
    ax[1].imshow(np.clip(sharpened, 0, 1))
    ax[1].set_title("Sharpened")
    for a in ax:
        a.axis("off")
    plt.tight_layout()
    plt.show()
```

Código-fonte 6 - Melhoria de nitidez
Fonte: Elaborado pelo autor (2025)

Na figura a seguir, temos a imagem original (uma imagem de astronauta sem modificações) e a imagem processada (aumento de nitidez usando um kernel que enfatiza o pixel central e subtrai os vizinhos).



Figura 23 - Melhorando a nitidez de imagens
Fonte: Elaborado pelo autor (2025)

8.3 Filtragem no domínio da frequência para remover padrões

Vamos simular interferências periódicas, como ruído elétrico, adicionando padrões de onda senoidal à imagem do astronauta. Em seguida, removemos esses padrões no domínio da frequência usando FFT e um filtro de entalhe.

```
    from numpy.fft import fft2, ifft2, fftshift, ifftshift
    from skimage.color import rgb2gray

    gray = rgb2gray(image)

    # Add synthetic periodic pattern (e.g., horizontal stripes)
    def add_stripe_noise(img, freq=10):
        rows, cols = img.shape
        sinusoid = 0.2 * np.sin(2 * np.pi * freq * np.arange(cols) / cols)
        stripe = np.tile(sinusoid, (rows, 1))
        return np.clip(img + stripe, 0, 1)


    noisy_freq = add_stripe_noise(gray)

    # FFT
    f = fft2(noisy_freq)
    fshift = fftshift(f)
    fshift[245:270, 245:270] = 0  # crude notch filter
    f_ishift = ifftshift(fshift)
```

```
restored = np.abs(ifft2(f_ishift))

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].imshow(noisy_freq, cmap="gray")
ax[0].set_title("With Stripe Noise")
ax[1].imshow(restored, cmap="gray")
ax[1].set_title("After FFT + Filtering")
for a in ax:
    a.axis("off")
plt.tight_layout()
plt.show()
```

Código-fonte 7 - Remoção de padrões indesejáveis por meio de processamento no domínio da frequência
Fonte: Elaborado pelo autor (2025)

O código anterior recebe a imagem da astronauta com um padrão de listras sintéticas e a processa no domínio da frequência. Assim, obtemos como resultado uma nova imagem restaurada usando um filtro de corte de frequência.

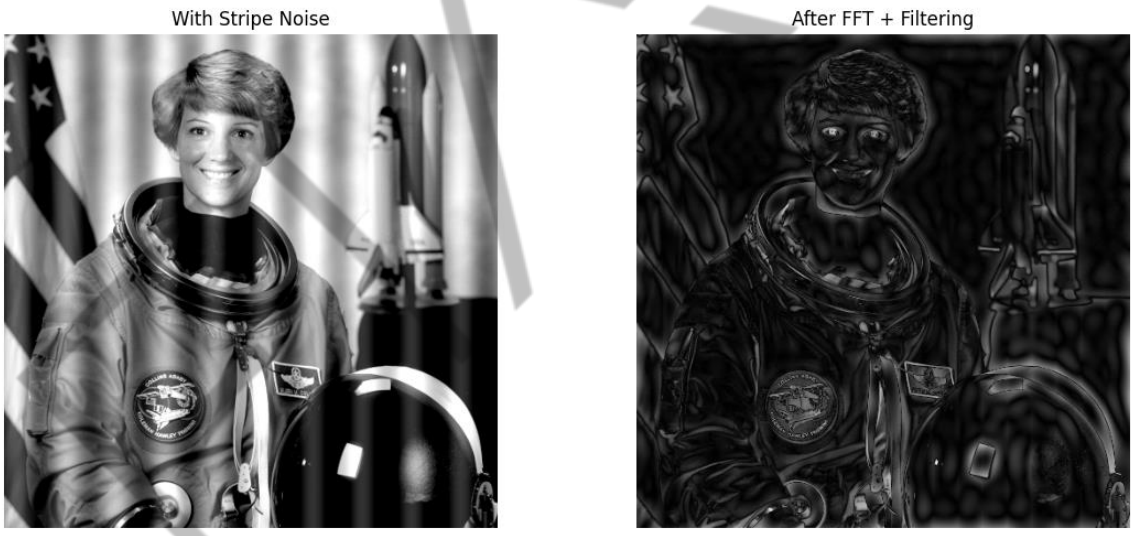


Figura 24 - Melhorando imagens por meio de processamento no domínio da frequência
Fonte: Elaborado pelo autor (2025)

CONCLUSÃO

Ao longo do capítulo, exploramos o universo dos filtros de imagens — desde os conceitos matemáticos fundamentais até as aplicações práticas em diferentes contextos, como Visão Computacional, medicina, fotografia e indústria. Aprendemos como filtros lineares e não lineares, aplicados no domínio espacial ou na frequência, são capazes de suavizar ruídos, destacar bordas, preservar detalhes importantes e até transformar a estética de uma imagem.

Ao utilizar a imagem da astronauta como exemplo recorrente, foi possível visualizar de forma concreta os efeitos de cada técnica, reforçando a importância de compreender e dominar esses recursos. O filtro de imagens está longe de ser apenas uma etapa preliminar — trata-se de uma poderosa ferramenta para análise, extração de informação e tomada de decisão visual, sendo essencial para qualquer profissional ou estudante que deseje atuar de forma sólida nas áreas de processamento de imagens e Visão Computacional.

REFERÊNCIAS

BRADSKI, G. **The OpenCV Library**. Dr. Dobb's Journal of Software Tools. 2000.

CANNY, J. **A Computational Approach to Edge Detection**. *In*: IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986. DOI: 10.1109/TPAMI.1986.4767851.

COUTO, G. Segmentação baseada em bordas – Sobel e Canny. **LinkedIn**, 14 de março de 2024. Disponível em: <https://pt.linkedin.com/pulse/segmenta%C3%A7%C3%A3o-baseada-em-bordas-sobel-e-canny-gabriel-couto-tlhhf>. Acesso em: 26 de jun. 2025.

DENG, J. K. LI, M. DO, H. SU, L. Fei-Fei, Construction and Analysis of a Large Scale Image Ontology. *In*: **Vision Sciences Society** (VSS), 2009.

ENGLISH Machine Learning in Plain. Convolutional Neural Network – Lesson 6: Padding. **Medium**, 18 de junho de 2023. Disponível em: <https://medium.com/@nerdjock/convolutional-neural-network-lesson-5-padding-98b50660ddbd>. Acesso em: 26 jun. 2025.

GIOVANINI, Adenilson. Filtragem de Imagens?. **Blog**, [2025?]. Disponível em: <https://adenilsongiovanini.com.br/blog/filtragem-de-imagens-o-que-e-e-tipos-existentes/>. Acesso em: 27 jun. 2025.

GONZALEZ, R. C.; WOODS, R. E. **Digital image processing**. Prentice Hall. 1992.

HE, Kaiming; SUN, Jian; TANG, Xiaoou. Guided Image Filtering. **IEEE Computer Society**, v. 35, p. 1397-1409, jun. 2013. Disponível em: <https://www.computer.org/csdl/journal/tp/2013/06/ttp2013061397/13rRUxYrbNs>. Acesso em: 27 jun. 2025.

JANEIRO, J. M. Gentle Introduction to CNN. **Medium**, 26 de Janeiro de 2021. Disponível em: <https://joaomariajaneiro.medium.com/introduction-to-cnn-ec3473366bbc>. Acesso em: 26 jun. 2025.

JUNOR CARVALHO, Arnaldo de. O Poder Das CNNs Em Aplicações de ML Envolvendo Identificação e Classificação de Imagens. **EAILab**, 13 de agosto de 2024.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. ImageNet classification with deep convolutional neural networks. **Communications of the ACM**. V. 60, n. 6, p. 84–9020, 17 de maio de 2024.

LECUN, Yann; BENGIO, Yoshua. Convolutional networks for images, speech, and time series. **The handbook of brain theory and neural networks**. 3361.10 (1995): 1995.

LOWE, David G. Distinctive Image Features from Scale-Invariant Keypoints. 2004. **International Journal of Computer Vision**, v. 60, n. 2, p. 91–110, November 2004.

NAG, H. Applications of Linear Algebra in Image Filters [Part II]- Separable Filters. **Medium**, 21 de junho de 2020. Disponível em: <https://naghemanth.medium.com/applications-of-linear-algebra-in-image-filters-part-ii-separable-filters-f4635ea9f94f>. Acesso em: 26 jun. 2025.

NETTO, G. G. Blur Filters: uma diferente aplicação. **Medium**, 31 de agosto de 2019. Disponível em: <https://medium.com/@ggaighernt/blur-filters-uma-diferente-aplica%C3%A7%C3%A3o-25d63c0f03c0>. Acesso em: 26 jun. 2025.

NIKOUKHAH, T. *et al.* Local JPEG Grid Detector via Blocking Artifacts, a Forgery Detection Tool. Image Processing On Line [**ResearchGate**], May 2020. Disponível em: <https://www.researchgate.net/publication/341552261_Local_JPEG_Grid_Detector_via_Blocking_Artifacts_a_Forgery_Detection_Tool>. Acesso em: 27 jun. 2025.

PHAM D. T. Kringing-Weighted Laplacian Kernels for Grayscale Image Sharpening. **IEEE Xplore**, v. 10, 2022. Disponível em: <https://ieeexplore.ieee.org/document/9783180/>. Acesso em: 27 jun. 2025.

SAN, C. T. Spatial Vs Frequency Domain: A Guide to Image Interpretation. **Medium**, 22 de novembro de 2024. Disponível em: <https://medium.com/@chawthirisan/spatial-vs-frequency-domain-a-guide-to-image-interpretation-d9c16b129b3f>. Acesso em: 27 jun. 2025.

SEDDIK, H. A new family of Gaussian filters with adaptive lobe location and smoothing strength for efficient image restoration. EURASIP Journal on Advances in Signal Processing [**ResearchGate**], March 2014. disponível em: <https://www.researchgate.net/publication/346997614_A_new_family_of_Gaussian_filters_with_adaptive_lobe_location_and_smoothing_strength_for_efficient_image_restoration>. Acesso em: 26 jun. 2025.

SIMONYAN, Karen; ZISSERMAN, Andrew. **Very deep convolutional networks for large-scale image recognition**. arXiv preprint arXiv:1409.1556. 2014.

WELLINGTONVCASTRO. DIP03 – Matriz de convolução e detecção de bordas, **CapivaraRex**, 25 de abril de 2016. Disponível em: <https://capivararex.wordpress.com/2016/04/25/dip03-matriz-de-convolucao-e-deteccao-de-bordas/>. Acesso em: 26 jun. 2025.